

Teoría de la Computabilidad

Módulo 20 Introducción a la Complejidad de Algoritmos

Prof. Carlos Iván Chesñevar

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Una introducción a la complejidad de algoritmos

Las transparencias que siguen se basan parcialmente en material de cursos sobre complejidad algorítmica.

Fuentes: Curso “Algoritmos y Complejidad”. Varias de las transparencias fueron elaboradas por los Profs. Marcelo Falappa, Pablo Fillottrani y Diego Martínez

Análisis de Algoritmos - Algoritmia

- La **algoritmia** es el estudio del diseño y análisis de algoritmos.
- Los problemas centrales de estos estudios son:
 - **Correctitud de los algoritmos:**
 - Finalización
 - Obtención del resultado deseado
 - **Eficiencia de los algoritmos:**
 - Cantidad de recursos que utiliza (tiempo, espacio)

Análisis de Algoritmos

Parte de estos estudios ya fueron planteados por diversos investigadores del concepto de computación muchos años antes...

As soon as an analytical engine exists, it will necessarily guide the future course of the science. Whenever any result is sought by its aid, the question will then arise:

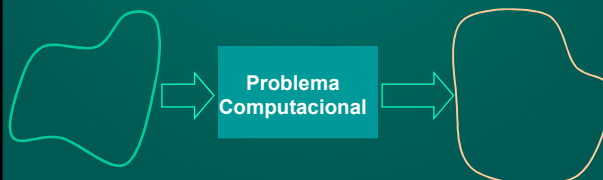
By what course of calculation can these results be arrived at by the machine in the shortest time?

Charles Babbage, 1864

Veamos algunas definiciones previas a fin de analizar la complejidad de los algoritmos...

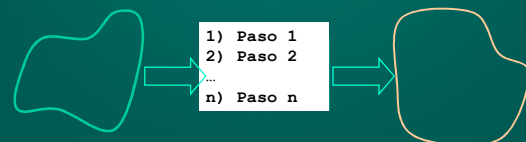
Análisis de Algoritmos - definiciones

Un **problema computacional** consiste en una caracterización de un conjunto de datos de entrada, junto con una especificación de la salida deseada en base a cada entrada.



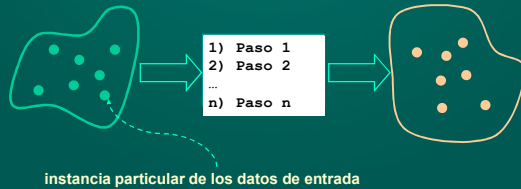
Análisis de Algoritmos - definiciones

Un **algoritmo** es una secuencia bien determinada de acciones elementales que transforma los datos de entrada en datos de salida con el objeto de resolver un problema computacional.



Análisis de Algoritmos - definiciones

Un problema computacional tiene una o más **instancias**, valores particulares de los datos de entrada, sobre las cuales se puede ejecutar un algoritmo para resolver el problema



Análisis de Algoritmos

Un problema computacional **abarca** a otro problema si las instancias del segundo pueden ser resueltas como instancias del primero en forma directa

Por ejemplo, **multiplicar n por 350** es un problema computacional abarcado por el problema de **multiplicar dos números enteros**.

Análisis de Algoritmos - Factores

Dos recursos importantes al momento de analizar la complejidad algorítmica: el **tiempo** y el **espacio**.

Problema: medirlos objetivamente:

- Para analizar el tiempo podría ejecutar el algoritmo sobre una computadora y medir el tiempo de ejecución...
Pero esto es relativo a una computadora en particular.
- Para analizar el tiempo podría contar el número de operaciones ejecutadas...
Pero esto es relativo a un lenguaje determinado.

Análisis de Algoritmos - factores

En realidad existen dos formas de analizar la eficiencia:

- **Empíricamente**: se programa el algoritmo y se corren tests con distintas entradas.
- **Teóricamente**: se determina matemáticamente la cantidad de recursos necesarios como una función de la entrada.

Análisis de Algoritmos - factores

Es claro que para cada algoritmo, la cantidad de recursos (memoria, tiempo) depende fuertemente del tamaño de la entrada.

Se definen entonces funciones de **cantidad de recursos** en base al **tamaño de la entrada** (cantidad de dígitos de un número, cantidad de elementos en un vector, etc).

Estas funciones nos determinan una “unidad de medida” de los factores de complejidad algorítmica.

Análisis de Algoritmos

Dado un algoritmo A , el tiempo de ejecución $t_A(n)$ de A es la cantidad de pasos, operaciones o acciones elementales que debe realizar el algoritmo al ser ejecutado en una instancia de tamaño n .

El espacio $e_A(n)$ de A es la cantidad de datos elementales que el algoritmo necesita al ser ejecutado en una instancia de tamaño n , sin contar la representación de la entrada.

Estas definiciones son ambiguas en dos sentidos:

- No se especifica claramente cuáles son las operaciones o datos elementales.
- Dado que existe más de una instancia de tamaño n no está claro cuál de ellas es la que se tienen en cuenta para el análisis.

Análisis de Algoritmos

Para resolver este último punto se definen varios tipos de análisis:

- **Análisis en el Peor Caso:** se considera el máximo entre las cantidades de recursos insumidos por todas las instancias de tamaño n .
- **Análisis en el Caso Promedio:** se considera el promedio entre las cantidades de recursos insumidos por todas las instancias de tamaño n .
- **Análisis Probabilístico:** se considera la cantidad de recursos de cada instancia de tamaño n , pesada por su probabilidad de ser ejecutada.
- **Análisis en el mejor caso:** se considera el mínimo entre las cantidades de recursos insumidos por todas las instancias de tamaño n .

Análisis de Algoritmos

Por lo general nos concentraremos en analizar el **peor caso**, pues:

- establece una cota superior al total de los recursos insumidos por el algoritmo.
- No se necesita conocer la distribución de probabilidades y por lo tanto es más sencillo.
- En muchas situaciones el peor caso es el más probable.

El tiempo de ejecución de un algoritmo se analiza teniendo en cuenta el costo de sus operaciones elementales, los ciclos repetitivos y la recursión.

Operaciones elementales

Una **operación elemental** es aquella cuyo tiempo de ejecución está acotado por una constante que depende sólo de la implementación usada.

No depende del tamaño de la entrada, por lo tanto sólo interesa el número de operaciones elementales.

En general, las sumas, multiplicaciones y asignaciones son operaciones elementales.

Algunas operaciones no pueden considerarse elementales, por ejemplo:

$x := \min\{T[i] \text{ tal que } 0 < i < n+1\}$

que busca el mínimo elemento en un arreglo de n componentes.

Operaciones elementales - ejemplo

	costo	veces
Proc select ($T[1..n]$)		
for $i:=1$ to $n-1$ do	C_0	$n-1$
$\text{minj} := i$	C_1	$n-1$
$\text{minx} := T[i]$	C_2	$n-1$
for $j:=i+1$ to n do	C_3	$n-(i+1)$
if $T[j] < \text{minx}$	C_4	$n-(i+1)$
then $\text{minj}:=j$	C_5	$n-(i+1)$
$\text{minx}:=T[j]$		
$T[\text{minj}] := T[i]$	C_6	$n-1$
$T[i] := \text{minx}$	C_7	$n-1$

Tratabilidad de problemas computacionales

Objetivos

- El estudio de la eficiencia de los algoritmos no puede ser atacado con el argumento **"las computadoras del futuro serán miles de veces más rápidas y, por lo tanto, no importará que el algoritmo sea eficiente"**.
- Ese argumento es **falaz** ya que, sin una mejora algorítmica, el problema del viajante para cien ciudades toma cien millones de años; la factorización de un número de 300 dígitos toma un millón de años.
- Por lo tanto, se necesitan mejoras en los algoritmos pero no pequeñas sino en los órdenes de magnitud.
- Por caso, la **búsqueda secuencial** toma un tiempo en $\Theta(n)$ mientras que la **búsqueda binaria** lo hace en $\Theta(\log n)$. Para $n = 10^9$, la diferencia está en esperar **un año** o **un minuto** respectivamente para cada algoritmo.

Objetivos

- Dado un algoritmo que resuelve un problema, es razonable (y aconsejable) preguntarse si no existirá un algoritmo más eficiente.
- El objetivo fundamental de la complejidad computacional es clasificar los problemas de acuerdo a su **tratabilidad**, tomando el o los algoritmos más eficientes para resolverlos.
- Deberíamos poder determinar:
¿Cuán tratable es el problema?
Si el problema es tratable: **¿el algoritmo es eficiente?**

Problemas Abiertos y Cerrados

- Si se demuestra que el problema no admite soluciones mejores, entonces se puede afirmar que el algoritmo es eficiente (salvo las constantes ocultas).
- Un problema se dice **cerrado** si se han encontrado algoritmos que lo resuelven y se ha demostrado que esos algoritmos son óptimos en cuanto al orden exacto del tiempo de ejecución.
- Por ejemplo, la **búsqueda en un arreglo ordenado** y el **ordenamiento de un arreglo** son problemas cerrados.

Grados de Tratabilidad

- Para cerrar un problema se puede hacer:
 - Encontrar un algoritmo asintóticamente mejor que los que se conocen.
 - Demostrar una cota asintóticamente superior de las que se conocen.
- En general, los distintos grados de tratabilidad son muy subjetivos (varían mucho de acuerdo al modelo computacional, los recursos disponibles, las variantes de las estructuras de datos, etc).
- **Objetivo primario** del estudio de la complejidad: definir cuáles problemas son tratables y cuáles no, para luego considerar distintos grados de tratabilidad o intratabilidad.

Grados de Tratabilidad

- Muchos problemas computacionales son tratables: tienen solución para instancias grandes y una mejora algorítmica o en el hardware produce una gran ampliación en el conjunto de instancias que se pueden resolver.
- Algunos problemas son intratables: el Problema de las Torres de Hanoi, el Problema del Viajante; los mismos pueden resolverse solamente para instancias pequeñas.
- En general, aquellos problemas con **complejidad polinomial** son **tratables**, mientras que los problemas con **complejidad superpolinomial** (más allá de polinomial) son **intratables**.

Monkey Puzzle

- Un ejemplo de problema intratable es el **Monkey Puzzle**, una especie particular de rompecabezas.
- Si se tiene un tablero de $n \times n$ no se conoce un algoritmo mejor que probar todas las permutaciones posibles para resolverlo.
- Esto es, su tiempo está en $\Omega(n!)$ y para $n=5$, generando un millón de configuraciones por segundo, se tardaría 490.000 millones de años en resolverlo.

Tratabilidad vs. Intratabilidad

- La evidencia empírica (una especie de **Tesis de Turing-Church** para la complejidad) permite acordar que un problema que admite solución polinomial es **tratable** y en caso contrario es **intratable**.
- El límite de tratabilidad entre algoritmos polinomiales y superpolinomiales es arbitrario pero mayormente aceptado.
- Por ejemplo, 1.00001^n o $n^{\log_2 n}$ serían intratables mientras que n^{10000} o $100000000n$ serían tratables.
- El punto es que estas funciones “extremas” raramente aparecen en casos reales como tiempo de ejecución de algoritmos.

Tratabilidad vs. Intratabilidad

- Los algoritmos con tiempo en $O(n^k)$ son considerados de tiempo razonable a pesar del k y de las posibles variables ocultas.
- En cuanto al espacio se debe ser más cuidadoso ya que muchas veces un **espacio polinomial se torna intratable**.
- Además, dentro de la tratabilidad e intratabilidad, es posible distinguir varios niveles.
- Actualmente, se está estudiando extender el concepto de tratabilidad a **algoritmos probabilísticos** y a **algoritmos paralelos**.

Tratabilidad vs. Intratabilidad

Función \ n	10	50	100	300	1000
$5n$	50	250	500	1500	5000
$n \cdot \log_2 n$	33	282	665	2469	9966
n^2	100	2500	10000	90000	1000000
n^3	1000	125000	1000000	27000000	10^9
2^n	1024	16 dígitos	31 dígitos	91 dígitos	302 dígitos
$n!$	3628800	65 dígitos	161 dígitos	623 dígitos	Overflow
n^n	11 dígitos	85 dígitos	201 dígitos	744 dígitos	Overflow

Referencias: el número de protones en el universo es de 79 dígitos y los microsegundos transcurridos desde el Big Bang es de 24 dígitos.

Tratabilidad vs. Intratabilidad

Esto significa que si un algoritmo ejecuta n operaciones por microsegundo, el algoritmo daría:

Función \ n	10	20	50	100	300
n^2	1/10 ⁴ seg	1/2500 seg	1/400 seg	1/100 seg	9/100 seg
n^5	1/10 seg	3.2 segs	5.2 mins	2.8 hs	28.1 días
2^n	1/10 ³ seg	1 seg	35.7 años	400 billones de siglos	Overflow
n^n	2.8 hs	3.3 billones de años	Overflow	Overflow	Overflow

Referencia: el Big Bang ocurrió hace aproximadamente 15000 millones de años.

Se necesitan mejores algoritmos

Supongamos que en la actualidad tenemos una computadora en la que ejecutamos distintos algoritmos para resolver un mismo problema. Veremos cuál es la ventaja de adquirir una más poderosa...

Tiempo	Instancia posible de resolver con:		
	Computadora Actual	100 veces más veloz	1000 veces más veloz
$\Theta(n)$	a	$100 * a$	$1000 * a$
$\Theta(n^2)$	b	$10 * b$	$31.6 * b$
$\Theta(2^n)$	c	$6.64 * c$	$9.97 * c$

Esto demuestra que los problemas intratables no se resuelven comprando hardware: es una característica intrínseca del problema el hecho de que sólo se puedan resolver instancias pequeñas.

Algunos ejemplos comunes...

Tiempo	Ejemplo
$O(1)$ constante	Agregar un ítem al ppio. de una lista
$O(\log N)$ log	Búsqueda en un arreglo ordenado
$O(N)$ lineal	Búsqueda en un arreglo desordenado
$O(N \log N)$ $n \cdot \log n$	Ordenar n ítems usando <i>mergesort</i>
$O(N^2)$ cuadrático	Camino más corto entre dos nodos de un grafo
$O(2^N)$ exp.	Resolver Torres de Hanoi

Introducción a la Complejidad computacional

Complejidad

Hemos visto que existen problemas de decisión que no pueden ser solucionados por algoritmos.

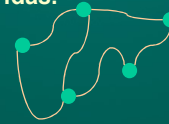
En función de esto, clasificamos los problemas de decisión en aquellos solucionables por algoritmos, y aquellos que no.

Desafortunadamente, la práctica indica que existen problemas que, aunque solubles por algoritmos, no son solubles en un sentido práctico debido a los **excesivos requerimientos de tiempo**.

Complejidad

Supongamos el siguiente problema conocido como “*Problema del viajante*”.

Debemos planificar las visitas a cierta cantidad de ciudades, de modo tal de minimizar las distancias recorridas.



Naturalmente, el número de itinerarios posibles es finito. Podemos construir una MT que solucione este problema

Complejidad

Si son n ciudades, los posibles itinerarios son $(n-1)!$

¡Son demasiados “tours” por examinar!

Si fuesen **10** ciudades, debemos examinar **$9! = 362880$** itinerarios. Es mucho, pero es una cantidad tratable.

Si fuesen **40** ciudades, debemos examinar **$39!$** itinerarios, que son más que **10^{45}**

Si pudiésemos examinar **10^{15}** por segundo (y ya es mucho), el tiempo requerido para finalizar el cálculo anterior es varias veces mayor que el tiempo de vida actual del universo!

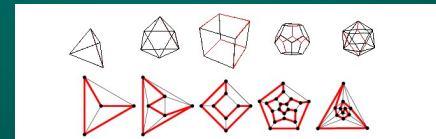
Ciclos hamiltonianos

Ciclo hamiltoniano: ciclo en un grafo donde cada nodo es visitado exactamente una vez.

Grafo hamiltoniano: grafo que tiene un ciclo de este tipo.



Sir William Rowan Hamilton (1805-1865)



Problema: supongamos que me dan un grafo G cualquiera ¿es G hamiltoniano?

Complejidad

Existe una teoría matemática formal, como refinamiento de la Tesis de Turing, que intenta determinar cuándo un algoritmo es tratable desde el punto de vista práctico.

Se clasifican los problemas de decisión de acuerdo al tiempo que tardaría una MT en resolver el problema (expresado como un lenguaje)

Sin duda son de interés aquellos dispositivos computacionales que se ejecutan un **número de pasos acotados por un polinomio basado en la longitud de la entrada (complejidad polinomial)**.

No importa que tipo de Máquina de Turing seleccionemos (con n cintas, de acceso directo, etc), siempre y cuando dejemos de lado el no-determinismo.

Complejidad - Clase P de problemas

Def: Una máquina de Turing M se dice **polinomialmente acotada**, si existe un polinomio $p(n)$, tal que para cualquier entrada x , **no existe** una configuración C tal que

$$(s_0, w, 1) \vdash p(|x|)+1 \ C$$

Esto es:

- hay una máquina de Turing M que es DETERMINISTA y recibe una entrada x (una secuencia de símbolos).
- La longitud de la entrada es $|x|$, y la usamos como dato de entrada para un polinomio (ej. $P(X) = x^2 + 2x^2 + 3$)
- En a lo sumo $p(|x|)$ pasos, la máquina siempre se detiene.

Complejidad - Clase P de problemas

Def: Una máquina de Turing M se dice **polinomialmente acotada**, si existe un polinomio $p(n)$, tal que para cualquier entrada x , no existe una configuración C tal que

$$(s_0, w, 1) \vdash p(|x|)+1 C$$

Un lenguaje se dice **polinomialmente decidable**, si es Turing-decidible con una máquina de Turing acotada polinomialmente.

P es la clase de lenguajes polinomialmente decidibles.

Informalmente, **P** es la clase de problemas que se resuelven en un tiempo polinomial por una MT *determinista*.

37

Complejidad - Clase P de problemas

Def: Una máquina de Turing M se dice **polinomialmente acotada**, si existe un polinomio $p(n)$, tal que para cualquier entrada x , no existe una configuración C tal que

$$(s_0, w, 1) \vdash p(|x|)+1 C$$

Un lenguaje se dice **polinomialmente decidable**, si es Turing-decidible con una máquina de Turing acotada polinomialmente.

P es la clase de lenguajes polinomialmente decidibles.

Informalmente, **P** es la clase de problemas que se resuelven en un tiempo polinomial por una MT *determinista*.

38

Complejidad - Clase P de problemas

Algunos problemas que parecen no pertenecer a P:

Ciclos de Hamilton: Dado un grafo G . ¿existe un ciclo que pasa a través de cada nodo de G exactamente una vez?

Conjunto Independiente: Dado un grafo no dirigido G y un entero $k \geq 2$. ¿existe un subconjunto C de nodos, con $|C| \geq k$ tal que para todo $v_i, v_j \in C$ no hay un arco entre v_i y v_j ?

Partición: Dado un conjunto C de enteros no negativos $a_1, a_2, \dots, a_n$ ¿existe una partición de C en conjuntos C_1 y C_2 tal que $\sum\{x|x \in C_1\} = \sum\{y|y \in C_2\}$?

Por ejemplo, para $\{38, 17, 52, 61, 21, 88, 25\}$ sí, pues $38+52+61=17+21+88+25=151$

39

Complejidad - Clase P de problemas

Aparentemente, el Problema del Viajante no se encuentra en esta categoría, ya que el tiempo para hallar la solución crece exponencialmente con la cantidad de ciudades del mapa.

Otros problemas similares **aparentemente no** pertenecen a la clase P de problemas (existencia de caminos en un grafo, satisfacibilidad de expresiones booleanas, etc)

A pesar de los constantes esfuerzos de encontrar una MT determinista que resuelva el Problema del viajante (entre otros) en tiempo polinomial, **ninguna se ha encontrado**.

Sin embargo, este problema, y otros similares, se pueden resolver en tiempo polinomial, pero con máquinas de Turing **no deterministas**.

40

Complejidad - Clase NP de problemas

Def: Una máquina de Turing M **no determinista** se dice **polinomialmente acotada**, si existe un polinomio $p(n)$, tal que para cualquier entrada x , no existe una configuración C tal que

$$(s_0, w, 1) \vdash p(|x|)+1 C$$

Esto es:

- hay una máquina de Turing M que es **NO DETERMINISTA** y recibe una entrada x (una secuencia de símbolos).
- La longitud de la entrada es $|x|$, y la usamos como dato de entrada para un polinomio (ej. $P(X) = x^3 + 2x^2 + 3$)
- En a lo sumo $p(|x|)$ pasos, la máquina siempre se detiene.

41

Complejidad - Clase NP de problemas

Def: Una máquina de Turing M **no determinista** se dice **polinomialmente acotada**, si existe un polinomio $p(n)$, tal que para cualquier entrada x , no existe una configuración C tal que

$$(s_0, w, 1) \vdash p(|x|)+1 C$$

Se define entonces otra clase de problemas:

NP es la clase de lenguajes decidibles con máquinas de Turing no deterministas polinomialmente acotadas.

Informalmente, **NP** es la clase de problemas que pueden ser resueltos por una MT *no determinista* en tiempo polinomial.

42

Complejidad – P versus NP

Claramente, $P \subseteq NP$.

Es decir, un problema que pertenezca a la clase P, también pertenece a la clase NP, pues las MT deterministas pueden considerarse MT no deterministas donde existe una sola opción en cada paso posible.

Aún no se sabe si $NP \subseteq P$.

Esto es, un problema perteneciente a la clase NP ¿puede pertenecer a la clase P?

Y por lo tanto, **no se pudo demostrar si $P=NP$.**

Es uno de los problemas abiertos más importantes de las ciencias de la computación actualmente.

43

Complejidad – P versus NP

Aún no se pudo demostrar si $P=NP$.

Es uno de los problemas abiertos más importantes de las ciencias de la computación actualmente.

El [Clay Mathematics Institute](#) (USA) ha ofrecido un premio de un millón de dólares para la persona que presente la primer demostración de este resultado.

44

Complejidad P versus NP

Algunos problemas que pertenecen a la clase NP poseen una importante propiedad de clausura: todos los problemas NP pueden reducirse a ellos vía reducciones de tiempo polinomial.

Estos problemas se denominan **NP-completos**.

Si bien estos problemas no permiten demostrar que $P \neq NP$, si efectivamente asumimos que $P \neq NP$ (algo generalmente aceptado) entonces todos los problemas NP-completos no están en P.

El concepto de reducción es el mismo que para las demostraciones de problemas insolubles, pero en este caso importa especialmente el tiempo de ejecución de la reducción.

45